

## **Relatório 11: Configuração de IDS de Rede usando Suricata no Debian (VM no VirtualBox)**

---

Este relatório descreve a configuração de um IDS com Suricata diretamente no Debian (uma única VM), monitorando o tráfego da própria VM para a Internet. Usaremos apenas regras locais (Custom Rules), sem baixar rulesets externos. O objetivo é instalar, iniciar o Suricata em modo IDS, gerar tráfego didático (via navegador/curl) e analisar os alertas no fast.log/eve.json.

- Material de apoio: <https://github.com/vinlss/T02---Redes-de-Dados-1/>

### **I. Introdução**

A única VM (Debian Desktop) atuará como sensor IDS (Suricata) e como cliente que gera tráfego. As regras locais foram desenhadas para disparar alertas de forma controlada e didática (URI, BODY/POST, HEADER e DNS).

Um IDS de rede opera de forma passiva: observa os pacotes, aplica assinaturas/regras de inspeção e gera alertas quando encontra padrões suspeitos, sem bloquear diretamente o tráfego. No Suricata, essa detecção ocorre com inspeção profunda de protocolos (DPI), permitindo visibilidade de elementos como URI, cabeçalhos, corpo HTTP e consultas DNS. Pilares: Confidencialidade e Monitoramento/Detecção, com ênfase em auditoria e visibilidade do tráfego.

### **II. Conceitos e Fundamentos**

- Firewall × IDS: firewall impõe política (permite/bloqueia); IDS observa e alerta (passivo).
- Suricata (DPI): inspeção por regras/assinaturas. Vamos criar regras locais para comprovar a capacidade de inspeção de URI, cabeçalhos, corpo HTTP e consultas DNS.
- HTTPS vs. HTTP: em HTTPS o conteúdo é cifrado (IDS vê metadados, p.ex. SNI/JA3); em HTTP o IDS consegue inspecionar cabeçalhos e corpo.
- Termos das regras (resumo):
  - flow:to\_server = tráfego do cliente para o servidor (requisição).
  - http.uri = caminho solicitado (p.ex. /rota).
  - http\_header = cabeçalhos HTTP (ex.: User-Agent, Host, cabeçalhos customizados).
  - http.request\_body = corpo enviado pelo cliente (POST/PUT).
  - dns.query = nome de domínio consultado no DNS.
  - classtype/priority = classificação e prioridade do alerta no log.

### III. Ambiente e Topologia

Topologia lógica (1 VM):

- user 1 - Debian (com Suricata) atuando como sensor IDS e cliente de teste.
- Saída para Internet através de NAT.

Topologia: user 1 - Debian ↔ NAT (VBox) ↔ Internet.

#### 1) Resumo do laboratório

| Elemento              | Definição                                            |
|-----------------------|------------------------------------------------------|
| Rede do VirtualBox    | NAT com DHCP do VirtualBox                           |
| VM do laboratório     | user 1 - Debian                                      |
| Papel da VM           | Sensor IDS com Suricata e cliente que gera o tráfego |
| Tráfego observado     | Saída da própria VM para a Internet                  |
| Evidências principais | Alertas em fast.log e eventos em eve.json            |
| Recursos da VM        | 6 GB de RAM e 3 núcleos de CPU                       |

#### 2) Referências usadas durante a execução

| Item                            | Uso                                                              |
|---------------------------------|------------------------------------------------------------------|
| Senha das máquinas virtuais     | pytest                                                           |
| Senha solicitada pelo sudo      | pytest                                                           |
| Interface de saída da VM        | Confirmada com ip route get 1.1.1.1                              |
| enp0s3                          | Nome usado nos exemplos do relatório para a interface monitorada |
| /etc/suricata/rules/local.rules | Arquivo das regras locais do experimento                         |
| /var/log/suricata/fast.log      | Log textual acompanhado durante os testes                        |
| /var/log/suricata/eve.json      | Log JSON usado para consulta alternativa dos alertas             |

#### 3) Pacotes usados no laboratório

- dnsutils, net-tools, curl, wget
- suricata, tcpdump, jq

#### 4) Guia auxiliar de VirtualBox

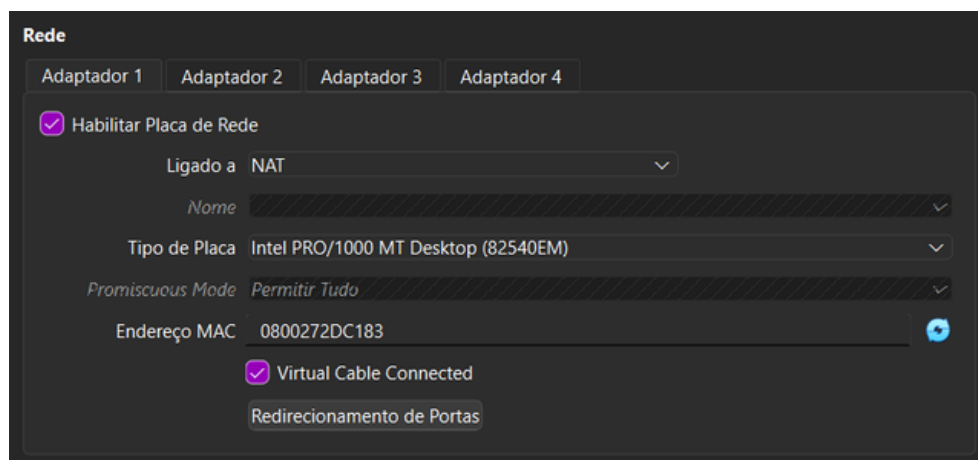
Se não estiver conseguindo copiar e colar comandos entre o computador host e a VM, redimensionar a tela corretamente ou usar melhor a integração com o VirtualBox, consulte o guia [VirtualBox Guest Additions.md](#) para instalar o VirtualBox Guest Additions.

### IV. Instalação e Preparação

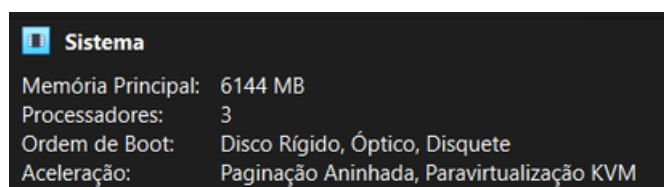
Se a interface indicada por ip route get 1.1.1.1 não for enp0s3, substitua apenas esse nome nos comandos de inicialização e captura que usam a interface de rede. Mantenha essa interface anotada antes de seguir, pois ela será usada na inicialização do Suricata e nos comandos de verificação do laboratório.

## 1) Preparar o ambiente no VirtualBox

- VM: user 1 - Debian
- Adapter 1: NAT (DHCP do VirtualBox)



- Recursos da VM: 6 GB de RAM e 3 núcleos de CPU



- Iniciar a VM

Ao final desta etapa: a VM deve estar configurada em NAT, com os recursos previstos e pronta para receber endereçamento via DHCP.

## 2) Após iniciar a VM, conferir a rede e padronizar o DHCP

### A) Padronizar a interface em DHCP

comando: `IFACE=$(ip route | awk '/default/ {print $5; exit}')`

`sudo dhclient -r "$IFACE" || true`

`sudo ip addr flush dev "$IFACE"`

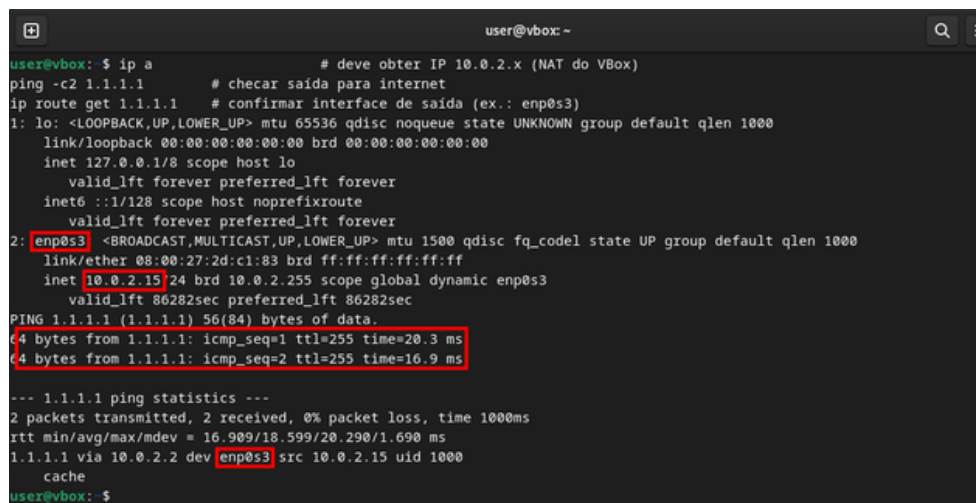
`sudo dhclient "$IFACE"`

`ip -4 a show "$IFACE"`

```
user@vbox:~$ IFACE=$(ip route | awk '/default/ {print $5; exit}')
sudo dhclient -r "$IFACE" || true
sudo ip addr flush dev "$IFACE"
sudo dhclient "$IFACE"
ip -4 a show "$IFACE"
[sudo] password for user:
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86400sec preferred_lft 86400sec
user@vbox:~$
```

## B) Validar IP, saída para Internet e interface monitorada

comando: `ip a` # deve obter IP 10.0.2.x (NAT do VBox)  
`ping -c2 1.1.1.1` # checar saída para internet  
`ip route get 1.1.1.1` # confirmar interface de saída (ex.: `enp0s3`)



```
user@vbox: ~$ ip a # deve obter IP 10.0.2.x (NAT do VBox)
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:2d:c1:83 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86282sec preferred_lft 86282sec
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data:
64 bytes from 1.1.1.1: icmp_seq=1 ttl=255 time=20.3 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=255 time=16.9 ms
--- 1.1.1.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1000ms
rtt min/avg/max/mdev = 16.909/18.599/20.290/1.690 ms
1.1.1.1 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 1000
    cache
user@vbox: ~$
```

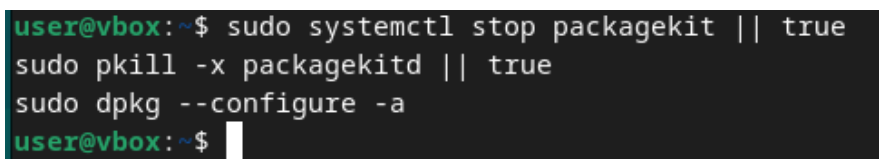
Resultado esperado: a VM deve apresentar um IP coerente com a rede NAT do VirtualBox, alcançar 1.1.1.1 e indicar a interface de saída usada pelo tráfego observado no laboratório.

Ao final desta etapa: a conectividade externa e a interface de monitoramento devem estar confirmadas antes da instalação do Suricata.

## 3) Preparar o sistema antes da instalação

### A) Prevenir lock do apt

comando: `sudo systemctl stop packagekit || true`  
`sudo pkill -x packagekitd || true`  
`sudo dpkg --configure -a`



```
user@vbox: ~$ sudo systemctl stop packagekit || true
sudo pkill -x packagekitd || true
sudo dpkg --configure -a
user@vbox: ~$
```

Ao final desta etapa: o sistema deve estar liberado para instalar os pacotes do laboratório sem conflito de gerenciador de pacotes.

## 4) Instalar utilitários e Suricata

comando: `sudo apt update && sudo apt install -y dnsutils net-tools curl wget suricata tcpdump jq`

```

user@vbox: ~$ sudo apt update && sudo apt install -y dnsutils net-tools curl wget suricata tcpdump jq
Hit:1 http://deb.debian.org/debian bookworm InRelease
Hit:2 http://security.debian.org/debian-security bookworm-security InRelease
Hit:3 http://deb.debian.org/debian bookworm-updates InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
271 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
dnsutils is already the newest version (1:9.18.47-1~deb12u1).
net-tools is already the newest version (2.10-0.1+deb12u2).
curl is already the newest version (7.88.1-10+deb12u14).
wget is already the newest version (1.21.3-1+deb12u1).
suricata is already the newest version (1:6.0.10-1).
tcpdump is already the newest version (4.99.3-1).
jq is already the newest version (1.6-2.1+deb12u1).
0 upgraded, 0 newly installed, 0 to remove and 271 not upgraded.
user@vbox: ~$

```

Ao final desta etapa: todas as ferramentas necessárias para gerar tráfego, monitorar a rede e consultar alertas devem estar instaladas.

## 5) Criar e validar as regras locais

### A) Entender o papel das regras

O que cada regra faz (antes do código):

- SID 1002001 (URI): alerta quando a URI da requisição HTTP contém /SURICATA\_BROWSER\_URI\_01. Demonstra inspeção de caminho (HTTP em claro).
- SID 1002002 (BODY/POST): alerta quando o corpo da requisição HTTP contém SURICATA\_BROWSER\_BODY\_02. Demonstra inspeção de payload em POST/PUT.
- SID 1002003 (HEADER): alerta quando um cabeçalho HTTP contém X-Trigger-Lab: 1. Demonstra inspeção de cabeçalhos (ex.: segurança, conformidade, políticas).
- SID 1002004 (DNS): alerta quando a consulta DNS contém suricata-trigger-lab.example. Demonstra visibilidade de DNS mesmo que o domínio não exista.

Arquivo: /etc/suricata/rules/local.rules (uma linha por regra)

### B) Criar o arquivo local.rules

em um único comando: “sudo tee /etc/suricata/rules/local.rules >/dev/null <<'RULES'

```

alert http any any -> any any (msg:"CUSTOM BROWSER - HTTP URI trigger";
flow:to_server; http.uri; content:"/SURICATA_BROWSER_URI_01"; nocase; sid:1002001;
rev:4; classtype:web-application-activity; priority:2;)

```

```

alert http any any -> any any (msg:"CUSTOM BROWSER - HTTP client body trigger";
flow:to_server; http.request_body; content:"SURICATA_BROWSER_BODY_02"; nocase;
sid:1002002; rev:4; classtype:web-application-attack; priority:2;)

```

```

alert http any any -> any any (msg:"CUSTOM BROWSER - HTTP header X-Trigger-Lab";
flow:to_server; content:"X-Trigger-Lab[3a] 1"; http_header; nocase; sid:1002003; rev:5;
classtype:attempted-recon; priority:2;)

```

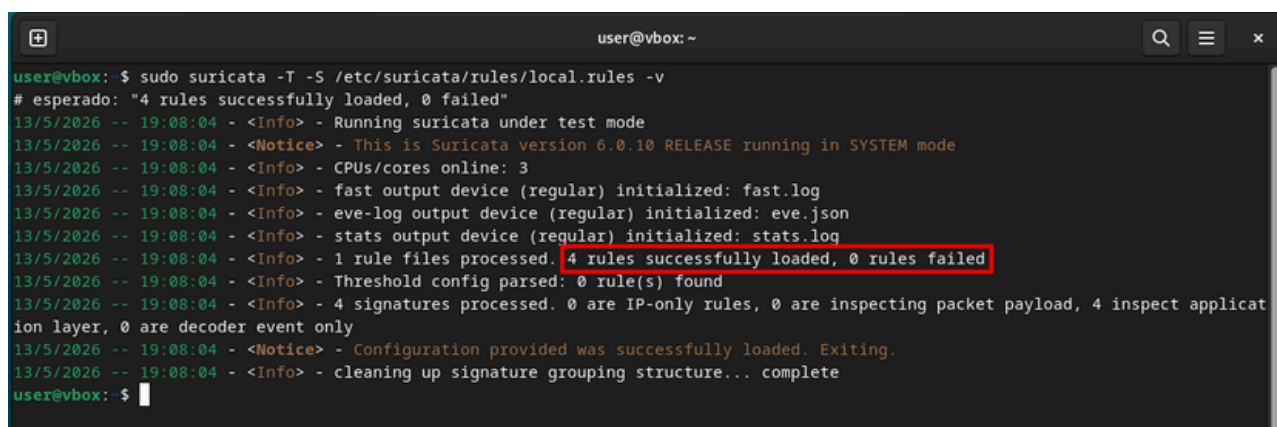
```
alert udp any any -> any 53 (msg:"CUSTOM BROWSER - DNS query trigger"; dns.query; content:"suricata-trigger-lab.example"; nocase; sid:1002004; rev:3; classtype:attempted-recon; priority:3;)
```

## RULES

```
user@vbox: $ sudo tee /etc/suricata/rules/local.rules >/dev/null <<'RULES'
alert http any any -> any any (msg:"CUSTOM BROWSER - HTTP URI trigger"; flow:to_server; http.uri; content:"/SURICATA_BROWSER_URI_01"; nocase; sid:1002001; rev:4; classtype:web-application-activity; priority:2;)
alert http any any -> any any (msg:"CUSTOM BROWSER - HTTP client body trigger"; flow:to_server; http.request_body; content:"SURICATA_BROWSER_BODY_02"; nocase; sid:1002002; rev:4; classtype:web-application-attack; priority:2;)
alert http any any -> any any (msg:"CUSTOM BROWSER - HTTP header X-Trigger-Lab"; flow:to_server; content:"X-Trigger-Lab|3a|1"; http_header; nocase; sid:1002003; rev:5; classtype:attempted-recon; priority:2;)
alert udp any any -> any 53 (msg:"CUSTOM BROWSER - DNS query trigger"; dns.query; content:"suricata-trigger-lab.example"; nocase; sid:1002004; rev:3; classtype:attempted-recon; priority:3;)
RULES
user@vbox: $
```

### C) Validar a sintaxe das regras

comando: `sudo suricata -T -S /etc/suricata/rules/local.rules -v`  
# esperado: "4 rules successfully loaded, 0 failed"



```
user@vbox: ~
user@vbox: $ sudo suricata -T -S /etc/suricata/rules/local.rules -v
# esperado: "4 rules successfully loaded, 0 failed"
13/5/2026 -- 19:08:04 - <Info> - Running suricata under test mode
13/5/2026 -- 19:08:04 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
13/5/2026 -- 19:08:04 - <Info> - CPUs/cores online: 3
13/5/2026 -- 19:08:04 - <Info> - fast output device (regular) initialized: fast.log
13/5/2026 -- 19:08:04 - <Info> - eve-log output device (regular) initialized: eve.json
13/5/2026 -- 19:08:04 - <Info> - stats output device (regular) initialized: stats.log
13/5/2026 -- 19:08:04 - <Info> - 1 rule files processed. 4 rules successfully loaded, 0 rules failed
13/5/2026 -- 19:08:04 - <Info> - Threshold config parsed: 0 rule(s) found
13/5/2026 -- 19:08:04 - <Info> - 4 signatures processed. 0 are IP-only rules, 0 are inspecting packet payload, 4 inspect application layer, 0 are decoder event only
13/5/2026 -- 19:08:04 - <Notice> - Configuration provided was successfully loaded. Exiting.
13/5/2026 -- 19:08:04 - <Info> - cleaning up signature grouping structure... complete
user@vbox: $
```

Ao final desta etapa: o arquivo de regras deve existir e o Suricata deve confirmar o carregamento correto das 4 regras locais.

### 6) Iniciar o Suricata em modo IDS e verificar a operação

Em Debian, usar `--set outputs.*` pode causar erro de "child node (null)". Usaremos o YAML padrão.

#### A) Subir o Suricata manualmente

```
# parar instâncias anteriores e limpar pid/log
sudo systemctl stop suricata
sudo pkill -x suricata || true
sudo rm -f /var/run/suricata.pid /var/log/suricata/fast.log
```



# subir em daemon na interface correta (ex.: enp0s3)# se sua interface tiver outro nome, descubra com: ip route get 1.1.1.1

sudo suricata -i enp0s3 -S /etc/suricata/rules/local.rules -l /var/log/suricata -D

# conferir que está ativo

ps aux | grep '[s]uricata'

sudo tail -n 15 /var/log/suricata/suricata.log

```
user@vbox:~$ # parar instâncias anteriores e limpar pid/log
sudo systemctl stop suricata
sudo kill -x suricata || true
sudo rm -f /var/run/suricata.pid /var/log/suricata/fast.log

# subir em daemon na interface correta (ex.: enp0s3)
# se sua interface tiver outro nome, descubra com: ip route get 1.1.1.1
sudo suricata -i enp0s3 -S /etc/suricata/rules/local.rules -l /var/log/suricata -D

# conferir que está ativo
ps aux | grep '[s]uricata'
sudo tail -n 15 /var/log/suricata/suricata.log
13/5/2026 -- 19:09:43 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
root      4245  0.0  0.1 54260 11112 ?        Rs   19:09   0:00 suricata -i enp0s3 -S /etc/suricata/rules/local.rules -l /var/
log/suricata -D
13/5/2026 -- 18:57:44 - <Error> - [ERRCODE: SC_ERR_FATAL(171)] - thread W#01-eth0 failed
13/5/2026 -- 19:08:04 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
13/5/2026 -- 19:08:04 - <Info> - CPUs/cores online: 3
13/5/2026 -- 19:08:04 - <Info> - fast output device (regular) initialized: fast.log
13/5/2026 -- 19:08:04 - <Info> - eve-log output device (regular) initialized: eve.json
13/5/2026 -- 19:08:04 - <Info> - stats output device (regular) initialized: stats.log
13/5/2026 -- 19:08:04 - <Info> - 1 rule files processed. 4 rules successfully loaded, 0 rules failed
13/5/2026 -- 19:08:04 - <Info> - Threshold config parsed: 0 rule(s) found
13/5/2026 -- 19:08:04 - <Info> - 4 signatures processed. 0 are IP-only rules, 0 are inspecting packet payload, 4 inspect applicat
ion layer, 0 are decoder event only
13/5/2026 -- 19:08:04 - <Notice> - Configuration provided was successfully loaded. Exiting.
13/5/2026 -- 19:08:04 - <Info> - cleaning up signature grouping structure... complete
13/5/2026 -- 19:09:43 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
13/5/2026 -- 19:09:43 - <Info> - CPUs/cores online: 3
13/5/2026 -- 19:09:43 - <Info> - Found an MTU of 1500 for 'enp0s3'
13/5/2026 -- 19:09:43 - <Info> - Found an MTU of 1500 for 'enp0s3'
user@vbox:~$
```

B) Confirmar onde os alertas serão acompanhados

Onde ver alertas: tail -f /var/log/suricata/fast.log

Alternativa JSON: jq 'select(.event\_type=="alert")' /var/log/suricata/eve.json | tail -n 5

Ao final desta etapa: o Suricata deve estar ativo em modo IDS, carregando local.rules e com os arquivos de log prontos para a observação dos testes.

## V. Procedimentos (Passo a Passo — user 1 - Debian ↔ Internet)

Checklist antes dos testes

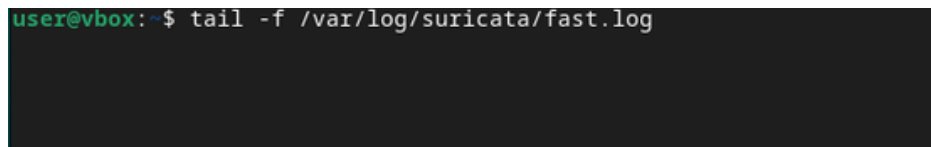
Antes de iniciar os cenários, confirme:

- as 4 regras locais foram validadas com suricata -T;
- o processo do Suricata está em execução;
- o terminal de acompanhamento do fast.log está aberto;
- a interface monitorada foi identificada previamente (ex.: enp0s3);
- a VM possui conectividade para gerar o tráfego HTTP e DNS do experimento.

## Organização dos terminais

Abra dois terminais:

Terminal A: `tail -f /var/log/suricata/fast.log`



```
user@vbox: ~$ tail -f /var/log/suricata/fast.log
```

Terminal B (caso optar por testar usando curl diretamente no terminal): comandos de disparo (abaixo). Dica: force IPv4 com `-4`.

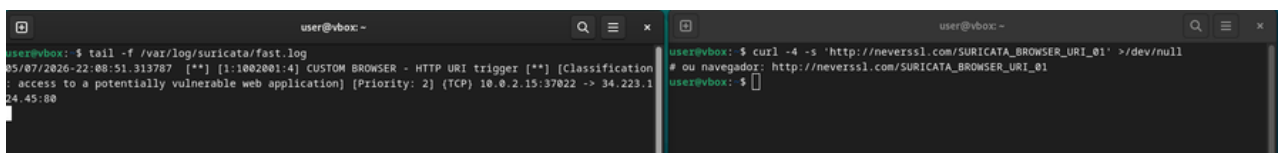
### Cenário 1 — HTTP URI (SID 1002001)

O que este teste demonstra: que o Suricata consegue inspecionar a URI (caminho) de uma requisição HTTP e gerar alerta quando encontra a string-alvo.

#### A) Executar o gatilho

comando: `curl -4 -s 'http://neverssl.com/SURICATA_BROWSER_URI_01' >/dev/null`  
# ou navegador: `http://neverssl.com/SURICATA_BROWSER_URI_01`

- Caso demore para executar ou alertar, aguarde. Este comportamento é esperado.



#### B) Observar o alerta no log

O que observar no log: uma linha no `fast.log` com a mensagem `CUSTOM BROWSER - HTTP URI trigger (SID 1002001)`, indicando tráfego `{TCP} <IP_VM>:<porta> -> <IP_destino>:80`.

Ao final deste cenário: deve existir evidência de que a regra de URI foi acionada pelo tráfego HTTP gerado.

### Cenário 2 — HTTP BODY/POST (SID 1002002)

O que este teste demonstra: que o Suricata consegue inspecionar o corpo (payload) de uma requisição HTTP (POST/PUT) e alertar quando a string estiver presente.

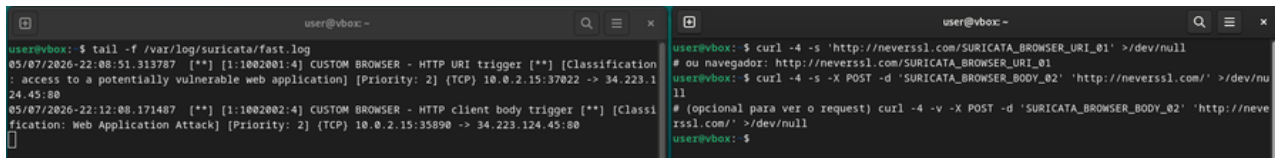
#### A) Executar o gatilho



comando:

```
curl -4 -s -X POST -d 'SURICATA_BROWSER_BODY_02' 'http://neverssl.com/' >/dev/null
# (opcional para ver o request) curl -4 -v -X POST -d 'SURICATA_BROWSER_BODY_02' 'http://neverssl.com/' >/dev/null
```

- Caso demore para executar ou alertar, aguarde. Este comportamento é esperado.

The image shows two terminal windows. The left window displays Suricata logs for two events: an HTTP URI trigger (SID 1002001) and an HTTP client body trigger (SID 1002002). The right window shows a series of curl commands being executed: first a GET request to SURICATA\_BROWSER\_URI\_01, then a POST request with SURICATA\_BROWSER\_BODY\_02, and finally a verbose POST request to check the request details.

## B) Observar o alerta no log

O que observar no log: a mensagem CUSTOM BROWSER - HTTP client body trigger (SID 1002002). Se não disparar de primeira, rode com -v e valide que o POST saiu; rode novamente.

Ao final deste cenário: deve existir evidência de que a regra de corpo da requisição foi acionada.

## Cenário 3 — HTTP HEADER (SID 1002003)

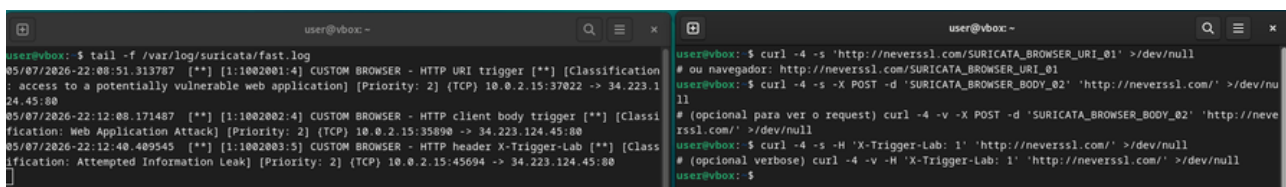
O que este teste demonstra: que o Suricata consegue inspecionar cabeçalhos HTTP e gerar alerta quando um cabeçalho customizado (X-Trigger-Lab: 1) está presente.

### A) Executar o gatilho

comando:

```
curl -4 -s -H 'X-Trigger-Lab: 1' 'http://neverssl.com/' >/dev/null
# (opcional verbose) curl -4 -v -H 'X-Trigger-Lab: 1' 'http://neverssl.com/' >/dev/null
```

- Caso demore para executar ou alertar, aguarde. Este comportamento é esperado.

The image shows two terminal windows. The left window displays Suricata logs for three events: an HTTP URI trigger (SID 1002001), an HTTP client body trigger (SID 1002002), and an HTTP header trigger (SID 1002003) for the X-Trigger-Lab header. The right window shows a series of curl commands: a GET request to SURICATA\_BROWSER\_URI\_01, a POST request with SURICATA\_BROWSER\_BODY\_02, and finally a verbose request with the X-Trigger-Lab: 1 header.

## B) Observar o alerta no log

O que observar no log: a mensagem CUSTOM BROWSER - HTTP header X-Trigger-Lab (SID 1002003), com fluxo {TCP} <IP\_VM>:<porta> -> <IP\_destino>:80.

Ao final deste cenário: deve existir evidência de que a regra de cabeçalho HTTP foi acionada.

## Cenário 4 — DNS (SID 1002004)

O que este teste demonstra: que o Suricata enxerga consultas DNS (nomes de domínio) e alerta quando a consulta contém o domínio-alvo.

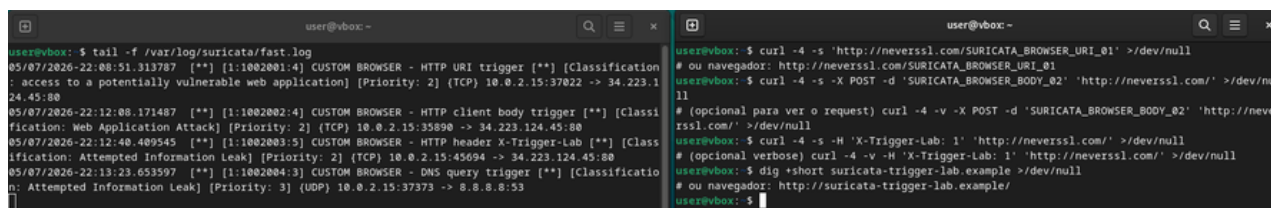
### A) Executar o gatilho

comando:

dig +short suricata-trigger-lab.example >/dev/null

# ou navegador: http://suricata-trigger-lab.example/

- Caso demore para executar ou alertar, aguarde. Este comportamento é esperado.



The image shows two terminal windows. The left window displays the output of 'tail -f /var/log/suricata/fast.log', showing several log entries for 'CUSTOM BROWSER' triggers, including a 'DNS query trigger' for SID 1002004. The right window shows a series of curl commands being executed to trigger the alert, including 'curl -4 -s 'http://neverssl.com/SURICATA\_BROWSER\_URI\_01'', 'curl -4 -s -X POST -d 'SURICATA\_BROWSER\_BODY\_02' 'http://neverssl.com/'', and 'dig +short suricata-trigger-lab.example >/dev/null'.

### B) Observar o alerta no log

O que observar no log: a mensagem CUSTOM BROWSER - DNS query trigger (SID 1002004), com fluxo {UDP} <IP\_VM>:<porta> -> <DNS\_resolveror>:53.

Ao final deste cenário: deve existir evidência de que a regra de consulta DNS foi acionada.

## VI. Verificação e Resultados

### 1) Checklist de sucesso do experimento

- suricata -T retorna 4 rules successfully loaded, 0 rules failed.
- O Suricata permanece em execução (ps aux | grep '[s]uricata') e sem erro crítico no suricata.log.
- Cada um dos quatro cenários gera o alerta correspondente em fast.log ou eve.json.

### 2) Quadro comparativo (cenários)

| Cenário | Tráfego                                  | Regra envolvida (msg)                      | SID     | Resultado esperado | Onde verificar      |
|---------|------------------------------------------|--------------------------------------------|---------|--------------------|---------------------|
| 1       | user 1 - Debian → Internet (HTTP/URI)    | CUSTOM BROWSER - HTTP URI trigger          | 1002001 | Alertas            | fast.log / eve.json |
| 2       | user 1 - Debian → Internet (HTTP/Body)   | CUSTOM BROWSER - HTTP client body trigger  | 1002002 | Alertas            | fast.log / eve.json |
| 3       | user 1 - Debian → Internet (HTTP/Header) | CUSTOM BROWSER - HTTP header X-Trigger-Lab | 1002003 | Alertas            | fast.log / eve.json |
| 4       | user 1 - Debian → Internet (DNS)         | CUSTOM BROWSER - DNS query trigger         | 1002004 | Alertas            | fast.log / eve.json |

### 3) Coleta rápida dos alertas

comando:

# Últimos alertas (formato fast)

```
tail -n 20 /var/log/suricata/fast.log
```

# Em JSON (EVE) com campos úteis, 1 por linha

```
jq -r 'select(.event_type=="alert") | "\(.timestamp) SID=\(.alert.signature_id) MSG=\(.alert.signature) \(.src_ip):\(.src_port) -> \(.dest_ip):\(.dest_port)'" \
/var/log/suricata/eve.json | tail -n 10
```

Use esta coleta rápida ao final dos cenários para consolidar a evidência dos alertas em formato textual ou JSON.

```
user@vbox: ~$ # Últimos alertas (formato fast)
tail -n 20 /var/log/suricata/fast.log

# Em JSON (EVE) com campos úteis, 1 por linha
jq -r 'select(.event_type=="alert") | "\(.timestamp) SID=\(.alert.signature_id) MSG=\(.alert.signature) \(.src_ip):\(.src_port) -> \(.dest_ip):\(.dest_port)'" \
/var/log/suricata/eve.json | tail -n 10
05/13/2026-19:15:07.214737 [**] [1:1002001:4] CUSTOM BROWSER - HTTP URI trigger [**] [Classification: access to a potentially vulnerable web application] [Priority: 2] (TCP) 10.0.2.15:45876 -> 34.223.124.45:80
05/13/2026-19:17:42.443866 [**] [1:1002002:4] CUSTOM BROWSER - HTTP client body trigger [**] [Classification: Web Application Attack] [Priority: 2] (TCP) 10.0.2.15:38776 -> 34.223.124.45:80
05/13/2026-19:17:56.905675 [**] [1:1002003:5] CUSTOM BROWSER - HTTP header X-Trigger-Lab [**] [Classification: Attempted Information Leak] [Priority: 2] (TCP) 10.0.2.15:59354 -> 34.223.124.45:80
05/13/2026-19:18:14.297999 [**] [1:1002004:3] CUSTOM BROWSER - DNS query trigger [**] [Classification: Attempted Information Leak] [Priority: 3] (UDP) 10.0.2.15:53614 -> 8.8.8.8:53
parse error: Invalid literal at line 327, column 2109
2026-05-05T14:13:54.883380-0300 SID=1002003 MSG=CUSTOM BROWSER - HTTP header X-Trigger-Lab 10.0.2.15:50330 -> 34.223.124.45:80
2026-05-05T14:15:01.900971-0300 SID=1002004 MSG=CUSTOM BROWSER - DNS query trigger 10.0.2.15:53382 -> 8.8.8.8:53
2026-05-05T14:17:59.730820-0300 SID=1002001 MSG=CUSTOM BROWSER - HTTP URI trigger 10.0.2.15:49464 -> 34.223.124.45:80
2026-05-05T14:18:28.659161-0300 SID=1002002 MSG=CUSTOM BROWSER - HTTP client body trigger 10.0.2.15:36540 -> 34.223.124.45:80
user@vbox: ~$
```

## VII. Conclusão

Com Suricata na própria VM e regras locais simples, comprovamos a detecção de URI, cabeçalhos, corpo HTTP e consultas DNS por meio de cenários objetivos e reproduzíveis. A configuração evita dependências externas e foca em didática, auditoria e visibilidade do tráfego HTTP/DNS.

### Apêndice — Troubleshooting rápido

- Sem alertas no fast.log:
  - a. Confirme que o Suricata está rodando: `ps aux | grep '[s]uricata'`
  - b. Veja o log de inicialização: `sudo tail -n 30 /var/log/suricata/suricata.log`
  - c. Verifique tráfego efetivo na interface: `sudo tcpdump -ni enp0s3 'port 80 or port 53' -c 10` (se necessário, substitua `enp0s3` pela interface indicada por `ip route get 1.1.1.1`)
- Erro “Failed to lookup configuration child node: (null)” ao iniciar: Não use `--set outputs.*` no Debian. Inicie assim: `sudo suricata -i enp0s3 -S /etc/suricata/rules/local.rules -l /var/log/suricata -D` (se a interface não for `enp0s3`, ajuste o nome da interface)
- pidfile “stale” / conflito de instância:

comando: `sudo systemctl stop suricata`

`sudo pkill -x suricata || true`

`sudo rm -f /var/run/suricata.pid`

- Regras não carregam / erros de sintaxe: Cada regra deve estar em uma única linha (Snort-syntax). Valide com `-T`.
- HTTP não dispara (HSTS/HTTPS forçado): Use `http://neverssl.com` e force IPv4 com `curl -4`.

- Confirmar que os triggers realmente estão saindo: Para POST/HEADER, rode em paralelo: `sudo tcpdump -A -ni enp0s3 'port 80' -c 10` e verifique o POST, o corpo e o cabeçalho X-Trigger-Lab em texto.
- Classificação diferente no log: Dependendo do mapeamento interno de classificação, `classtype:attempted-recon` pode aparecer como Attempted Information Leak no `fast.log`.